Lezione 10 – Eccezioni e ricorsione

`try`, `catch`, `finally`, `throw` e `throws`, eccezioni controllate e non, le nostre eccezioni; casi base e ricorsivi, lo stack delle chiamate

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

10 novembre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 9** (*15 min*)
   - uno di voi alla lavagna con `Spedibile`, `Termostato` o `Magazzino`: lo commentiamo insieme
2. **Le eccezioni** (*25 min*)
   - errori a tempo di esecuzione e stack trace; `try` e `catch`; `throw`
   - propagazione e rilancio; la gerarchia, `throws`, eccezioni controllate e non; `finally`; le nostre eccezioni
3. **La ricorsione** (*20 min*)
   - casi base e ricorsivi; il fattoriale; lo stack delle chiamate; terminazione
   - ricorsione o iterazione; da ricorsivo a iterativo
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: tre metodi ricorsivi, letti riga per riga
   - *Insieme, passo passo*: il conto bancario che segnala gli errori con le eccezioni
5. **Wrap-up e Homework 10** (*10 min*)

Eccezioni: errori a tempo di esecuzione – `Divisione.java`

```java
public class Divisione {

    public static int quoziente(int a, int b) {
        return a / b;
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Due interi: ");
        int a = in.nextInt();
        int b = in.nextInt();
        int q = quoziente(a, b);
        System.out.println("quoziente: " + q);
        System.out.println("fine");
    }
}
```

```
Due interi: 7 0
Exception in thread "main"
java.lang.ArithmeticException: / by zero
    at Divisione.quoziente(Divisione.java:7)
    at Divisione.main(Divisione.java:15)
```

Eccezione

Un'*eccezione* è un oggetto che la JVM crea quando un'operazione non può essere completata. L'esecuzione si interrompe e l'eccezione è **sollevata** e rimandata a chi ha chiamato; se nessuno la gestisce, il programma termina con lo *stack trace*.

- già incontrate: `ClassCastException` (lezione 7), `NullPointerException` (lezione 8)
- **lo stack trace**: classe e messaggio (`/ by zero`), poi una riga per metodo, con file e numero di riga: in cima dove è nata (`quoziente`, riga 7 del file), sotto chi l'ha chiamata, fino al `main`
- `fine` non viene stampato: anche `main` è terminato

Gestire un'eccezione: `try` e `catch` – `Divisione.java` (versione 2)

```
try {
    int a = in.nextInt();
    int b = in.nextInt();
    int q = quoziente(a, b);
    System.out.println("quoziente: " + q);
} catch (InputMismatchException e) {
    System.out.println("servono due interi");
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
}
System.out.println("fine");
```

```
Due interi: 7 0
/ by zero
fine
```

```
Due interi: 7 x
servono due interi
fine
```

`InputMismatchException` va importata da `java.util`, come `Scanner`; `ArithmeticException` sta in `java.lang`. `a`, `b` e `q` sono locali al blocco `try` (lezione 3): dopo non esistono più.

`try` e `catch`

Nel blocco `try` sta il codice che può sollevare un'eccezione; ogni `catch` dichiara il **tipo** che sa gestire e un parametro, `e`, riferimento all'oggetto eccezione. Se nel `try` viene sollevata un'eccezione, le istruzioni che seguono nel `try` sono saltate e viene eseguito il **primo** `catch` del tipo giusto; poi si prosegue dopo l'ultimo `catch`. Se non viene sollevato nulla, i `catch` sono ignorati.

- con `7 0`: la `println` del quoziente è saltata, si esegue il secondo `catch`, poi `fine`: il programma **non** termina. `e.getMessage()` è il messaggio; `println(e)` stampa classe e messaggio

Sollevare un'eccezione: `throw` – `Media.java`

```java
/** La media di v, che non deve essere vuoto. */
public static double media(int[] v) {
    if (v.length == 0) {
        throw new IllegalArgumentException(
                "media di un array vuoto");
    }
    int somma = 0;
    for (int x : v) {
        somma += x;
    }
    return (double) somma / v.length;
}
```

```java
public static void main(String[] args) {
    int[] voti = { 27, 30, 24 };
    int[] vuoto = {};
    System.out.println(media(voti));
    try {
        System.out.println(media(vuoto));
    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
    }
}
```

```
27.0
media di un array vuoto
```

throw

Un metodo che rileva una situazione che non sa gestire crea un oggetto eccezione con `new` e lo solleva con `throw`: il metodo **termina subito**, senza restituire nulla, e l'eccezione passa al chiamante. **Rileva** chi se ne può accorgere (il metodo); **gestisce** chi sa che cosa fare (il client).

- un `boolean` di esito non basta quando il metodo deve restituire un **valore**: nessun `double` può dire "media di niente" (`0.0` è una media legittima)
- `a / b` con `b` zero è come un `throw` di `ArithmeticException`, fatto dalla JVM. `IllegalArgumentException` sta in `java.lang`

Propagazione: lo stack unwinding – `Catena.java`

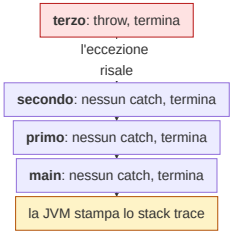
```java
public static void main(String[] args) {
    primo();
}

public static void primo() {
    secondo();
}

public static void secondo() {
    terzo();
}

public static void terzo() {
    throw new IllegalArgumentException("errore");
}
```

```
Exception in thread "main"
java.lang.IllegalArgumentException: errore
    at Catena.terzo(Catena.java:17)
    at Catena.secondo(Catena.java:13)
    at Catena.primo(Catena.java:9)
    at Catena.main(Catena.java:5)
```



Stack unwinding

Un'eccezione non catturata **risale la catena delle chiamate**: ogni metodo attraversato termina e viene rimosso dallo stack, finché un `catch` la cattura; se non la cattura nessuno, nemmeno il `main`, il programma termina.

Catturare lungo la catena e rilanciare – CatenaRilancio.java

```java
public static void main(String[] args) {
    primo();
    System.out.println("fine");
}

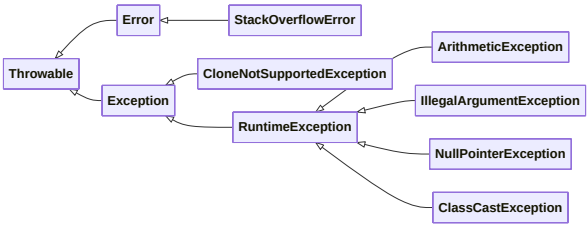
public static void primo() {
    try {
        secondo();
    } catch (IllegalArgumentException e) {
        System.out.println(e);
    }
}

public static void secondo() {
    try {
        terzo();
    } catch (IllegalArgumentException e) {
        System.out.println("secondo: rilancio");
        throw e;
    }
}
```

```
secondo: rilancio
java.lang.IllegalArgumentException: errore
fine
```

- il `catch` può stare in **qualunque** metodo della catena: `primo` cattura ciò che nasce in `terzo`, due livelli più in basso
- la risalita si ferma al primo `catch` del tipo giusto; da lì l'esecuzione **prosegue normalmente**: `primo` stampa l'eccezione (classe e messaggio) e termina come sempre, `main` stampa `fine`
- **rilancio**: `secondo` cattura, fa la sua parte (un messaggio) e rilancia **lo stesso oggetto** con `throw e`: l'eccezione riprende a risalire. `terzo` è quello di prima: chi la solleva non sa, e non deve sapere, chi la gestirà
- il tipo nel `catch` può essere una **superclasse** di quella sollevata (upcasting, lezione 7): qui `catch (RuntimeException e)` funzionerebbe

La gerarchia delle eccezioni: controllate e non controllate



Le eccezioni sono **oggetti di classi**, in una gerarchia (lezione 7) la cui radice è la classe `Throwable` (una classe, non un'interfaccia). Una parte: `InputMismatchException` e `ArrayIndexOutOfBoundsException` discendono da `RuntimeException`, `StackOverflowError` da `Error`, a più livelli.

Fra più `catch` vince il **primo** compatibile: la sottoclasse va scritta prima della superclasse.

Controllate e non controllate

Le sottoclassi di `RuntimeException` sono *non controllate* (*unchecked*): di solito errori di programmazione, che si evitano scrivendo bene il codice. Le altre sottoclassi di `Exception` sono *controllate* (*checked*): situazioni da prevedere; il **compilatore controlla** che chi può riceverne una la gestisca o la dichiari.

- `Error`: guasti gravi, come lo *stack overflow* (con la ricorsione): non controllati, non si dovrebbero catturare
- un `catch` cattura anche le **sottoclassi** del suo tipo: `catch (Exception e)` prende tutte le eccezioni, anche quelle che non ci aspettavamo: meglio il tipo più specifico

Dichiarare le eccezioni: `throws` – `ProvaClone.java`

In `Persona implements Cloneable`, come nella lezione 8:

```
@Override
public Persona clone()
        throws CloneNotSupportedException {
    return (Persona) super.clone();
}
```

Il client **cattura**, invece di dichiarare `throws` sul `main`:

```
public static void main(String[] args) {
    Persona p = new Persona("Anna", "Russo");
    try {
        Persona q = p.clone();
        System.out.println("copia: " + q);
    } catch (CloneNotSupportedException e) {
        System.out.println(e);
    }
}
```

```
copia: Anna Russo
```

Per le eccezioni non controllate `throws` non è obbligatorio: `media` non dichiara la sua.

`throws`

Nell'intestazione, `throws` elenca le eccezioni **controllate** che possono uscire dal metodo, sollevate da lui stesso o da un metodo che chiama, e che il metodo non cattura: le **rimanda al chiamante**. Chi lo chiama, a sua volta, deve **catturarle** (`try`/`catch`) oppure **rimandarle al proprio chiamante**, dichiarandole con `throws`.

- finalmente la riga della lezione 8: `Object.clone()` dichiara `throws CloneNotSupportedException` e la solleva se l'oggetto non è `Cloneable` (lezione 9). Senza `throws` né `catch`: *"unreported exception CloneNotSupportedException; must be caught or declared"*
- senza `implements Cloneable` si compila lo stesso, ma si arriva al `catch`:

```
java.lang.CloneNotSupportedException: Persona
```

finally e multi-catch – DivisioneFinally.java

```
try {
    int a = in.nextInt();
    int b = in.nextInt();
    System.out.println(a / b);
} catch (InputMismatchException
        | ArithmeticException e) {
    System.out.println("errore: " + e);
} finally {
    System.out.println("tentativo concluso");
}
System.out.println("fine");
```

```
Due interi: 7 0
errore: java.lang.ArithmeticException: / by zero
tentativo concluso
fine
```

```
Due interi: 7 2
3
tentativo concluso
fine
```

finally

Il blocco `finally`, dopo i `catch`, viene eseguito **sempre**: che il `try` finisca normalmente, che un `catch` gestisca l'eccezione, che l'eccezione sia rimandata al chiamante o che il `try` esegua un `return`.

- serve per il lavoro da fare **comunque**: ad esempio chiudere un file (lezione 13)
- **multi-catch**: un solo `catch` per più tipi separati da `|`, ammesso che abbiano una gestione comune. e ha il tipo dell'antenato comune più vicino: qui `RuntimeException`
- l'unico vincolo è fra i tipi **elencati**: nessuno può essere sottoclasse di un altro dell'elenco (`ArithmeticException | RuntimeException` non compila). Avere antenati in comune non conta: `StackOverflowError | InputMismatchException` è lecito, ed e è un `Throwable`

Le nostre eccezioni – ParcheggioPienoException.java

```java
/** Sollevata da entra(): parcheggio pieno. */
public class ParcheggioPienoException
        extends Exception {

    public ParcheggioPienoException() {
        super("parcheggio pieno");
    }
}
```

In Parcheggio (nome, posti, occupati):

```java
/** Occupa un posto libero. */
public void entra()
        throws ParcheggioPienoException {
    if (occupati == posti) {
        throw new ParcheggioPienoException();
    }
    occupati++;
}
```

Controllata: `p.entra()` senza `try` né `throws` non compila: *"unreported exception ParcheggioPienoException; must be caught or declared to be thrown"*.

```java
Parcheggio p = new Parcheggio("P1", 2);
try {
    p.entra();
    p.entra();
    p.entra();
    System.out.println("tre auto dentro");
} catch (ParcheggioPienoException e) {
    System.out.println(e.getMessage());
}
System.out.println(p);
```

```
parcheggio pieno
P1: 2/2
```

- un'eccezione nostra è una **sottoclasse** di `Exception` (controllata) o di `RuntimeException` (non controllata); il nome finisce per convenzione in `Exception`
- il costruttore passa il messaggio con `super(...)` (lezione 7): è quello che restituisce `getMessage()`. Può avere variabili di istanza e metodi, come ogni classe: nell'esercitazione

Quando (non) usare le eccezioni – `Quoziente.java`

```
int a = 7;
int b = 0;
// NO: l'eccezione usata come un if
try {
    System.out.println(a / b);
} catch (ArithmeticException e) {
    System.out.println("divisore nullo");
}
// SI': il controllo normale, con un if
if (b != 0) {
    System.out.println(a / b);
} else {
    System.out.println("divisore nullo");
}
```

Le due versioni stampano lo stesso messaggio; la seconda dice nell'`if` qual è il caso particolare, la prima lo nasconde in un `catch`.

- le eccezioni servono quando **chi rileva** il problema non sa come gestirlo e **chi sa** gestirlo non sa come rilevarlo: `media` sa che l'array è vuoto, ma solo il client sa se avvisare, ripetere o rinunciare
- **non** al posto del normale controllo di flusso: se basta un `if` (e un `boolean`), si usa l'`if`. Prima di dividere si controlla il divisore; prima di un cast, `instanceof` (lezione 8)
- **controllata o non controllata?** Controllata se il chiamante **deve** prevedere il caso e può rimediare (il parcheggio pieno: cercarne un altro). Non controllata se il chiamante l'avrebbe evitata scrivendo bene il codice: una precondizione violata, come l'array vuoto passato a `media`
- un'eccezione non gestita termina il programma: meglio pensarci prima

Ricorsione: definire qualcosa in termini di se stesso

Ricorsione

Una definizione è *ricorsiva* quando descrive una cosa usando la cosa stessa, applicata a un caso **più piccolo**: la definizione si riapplica al caso più piccolo, e poi a quello ancora più piccolo, finché si arriva a un caso così semplice da avere la risposta **diretta**. Un metodo è ricorsivo se, per risolvere un problema, chiama se stesso su un problema dello stesso tipo ma più piccolo.

- il **fattoriale**: $0! = 1$; per $n \geq 1$, $n! = n \cdot (n-1)!$: il fattoriale di n è definito con il fattoriale di $n-1$
- una **stringa** è la stringa vuota, oppure un carattere seguito da una stringa: `"java"` è `'j'` seguito da `"ava"` (`s.charAt(0)` e `s.substring(1)`, lezione 2)
- cercare una parola nel **dizionario**: si apre a metà e si cerca nella metà giusta, che è ancora un dizionario, più piccolo (la ricerca binaria, lezione 11)

Come si svolge un calcolo ricorsivo: $3!$

1. **si scende**, applicando la definizione finché compare un caso con risposta diretta:
   $3! = 3 \cdot 2!$ $\quad 2! = 2 \cdot 1!$ $\quad 1! = 1 \cdot 0!$ $\quad 0! = 1$
2. **si risale**, sostituendo i valori ottenuti, dall'ultimo al primo:
   $1! = 1 \cdot 1 = 1$ $\quad 2! = 2 \cdot 1 = 2$ $\quad 3! = 3 \cdot 2 = 6$

Ogni passo in discesa lascia in sospeso un calcolo $(3 \cdot ?)$, che si completa in risalita. Da qui gli ingredienti di un metodo ricorsivo:

- uno o più **casi base**: la risposta è diretta, senza chiamate ($0! = 1$; la stringa vuota)
- il **caso ricorsivo**: una chiamata su un problema **più piccolo**, il cui risultato si combina con il resto $(n \cdot (n-1)!)$
- la **convergenza**: ogni chiamata si avvicina a un caso base, altrimenti la discesa non finisce

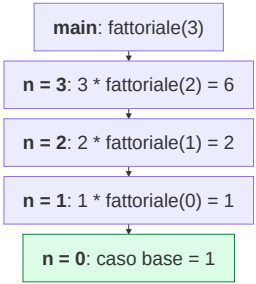
La ricorsione in Java: il fattoriale – `Fattoriale.java`

```java
/** n!, con 0 <= n <= 12. */
public static int fattoriale(int n) {
    if (n < 0) {
        throw new IllegalArgumentException(
                "n negativo: " + n);
    }
    if (n == 0) {
        return 1;        // caso base
    }
    // caso ricorsivo
    return n * fattoriale(n - 1);
}

public static void main(String[] args) {
    System.out.println(fattoriale(3));
    System.out.println(fattoriale(12));
}
```

```
6
479001600
```

$13! = 6\,227\,020\,800$ non sta in un `int`: da qui la precondizione $n \leq 12$. Un solo codice, tante attivazioni: come un metodo che ne chiama un altro.



- **fase discendente** (dall'alto): ogni chiamata aspetta la successiva; lo stack cresce a ogni chiamata, ciascuna con il **suo** `n` (lezione 3)
- **fase ascendente** (dal basso): il caso base restituisce 1; ogni chiamata moltiplica, restituisce e viene rimossa dallo stack

Terminazione e StackOverflowError – SenzaBase.java

```java
/** Nessun controllo su n < 0. */
public static int fattoriale(int n) {
    if (n == 0) {
        return 1;
    }
    return n * fattoriale(n - 1);
}

public static void main(String[] args) {
    System.out.println(fattoriale(-1));
}
```

```
Exception in thread "main"
java.lang.StackOverflowError
    at SenzaBase.fattoriale(SenzaBase.java:9)
    at SenzaBase.fattoriale(SenzaBase.java:9)
    at SenzaBase.fattoriale(SenzaBase.java:9)
    ... (1024 righe at in tutto)
```

Terminazione

Un metodo ricorsivo termina se ogni catena di chiamate arriva a un caso base: a ogni chiamata il problema deve **diminuire**, verso un caso base che prima o poi viene raggiunto. Il numero di chiamate annidate è la *profondità* della ricorsione.

- con $n = -1$: -2, -3, ...: n diminuisce ma **si allontana** da 0. Ogni chiamata occupa spazio sullo stack, che ha una dimensione **finita**: quando è pieno, la JVM solleva `StackOverflowError`, un `Error`: non va catturato, va corretto il codice
- è l'equivalente ricorsivo del ciclo infinito; con un ciclo, però, il programma resta bloccato, qui termina con un errore
- il rimedio è nella versione della slide precedente: la precondizione $n \geq 0$ controllata con un `throw`

Ricorsione o iterazione? – `ContoAllaRovescia.java`

```java
public static void contoAllaRovescia(int n) {
    if (n == 0) {
        System.out.println("via!");
    } else {
        System.out.println(n);
        contoAllaRovescia(n - 1);
    }
}

public static void contoIterativo(int n) {
    while (n != 0) {
        System.out.println(n);
        n = n - 1;
    }
    System.out.println("via!");
}
```

Con 3, entrambi stampano 3, 2, 1, via!

- lo stesso problema, risolto in due modi: nella versione ricorsiva il "ripeti" è la chiamata a se stesso e l'arresto è il caso base; in quella iterativa il "ripeti" è il `while` e l'arresto è la sua condizione
- ogni metodo ricorsivo ha una versione iterativa: qui basta un `while`, per il fattoriale un `for` che accumula il prodotto; per altri problemi (la Torre di Hanoi dell'homework) è molto più complicata
- **quando usarla**: se il problema è ricorsivo per natura e la soluzione ricorsiva è più semplice e leggibile; l'iterazione se il ciclo è ovvio e le prestazioni contano

	iterazione	**ricorsione**
costrutto	`while`, `for`	`if` + chiamata
si ferma	condizione del ciclo	caso base
se non si ferma	ciclo infinito	`StackOverflowError`
costo	nessuna chiamata	una chiamata per passo

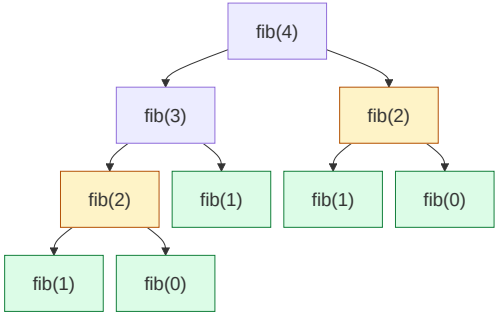
Commentiamo insieme (20 min) – il problema

Il problema. Tre piccoli problemi da risolvere **senza cicli**, solo con la ricorsione. Il codice è già scritto, nella classe `Ricorsivi`: lo leggiamo riga per riga cercando, in ogni metodo, il caso base, il caso ricorsivo e perché la ricorsione termina.

I tre problemi

1. la **somma delle cifre** di un intero non negativo: per 1234, $1 + 2 + 3 + 4 = 10$. L'ultima cifra è `n % 10`, il numero senza l'ultima cifra è `n / 10`
2. la **stampa inversa** di un array: per `{ 3, 8, 1, 6 }`, `6 1 8 3`, senza indici che scendono
3. i **numeri di Fibonacci**: $F_0 = 0$, $F_1 = 1$ e ciascuno dei successivi è la somma dei due precedenti: 0, 1, 1, 2, 3, 5, 8, 13, ... Il programma conta anche quante chiamate servono per calcolare F_{20}

Le chiamate per calcolare F_4



In verde i casi base; in giallo lo stesso calcolo, fatto due volte.

Commentiamo insieme – sommaCifre e stampaInversa

Nella classe `Ricorsivi`:

```
private static int chiamate = 0;

/** La somma delle cifre di n, n >= 0. */
public static int sommaCifre(int n) {
    if (n < 10) {
        return n;
    }
    return n % 10 + sommaCifre(n / 10);
}
```

- caso base: un numero di una cifra; caso ricorsivo: l'ultima cifra più la somma delle cifre del resto
- `sommaCifre(1234)` = `4 + sommaCifre(123)` = `4 + 3 + sommaCifre(12)` = `4 + 3 + 2 + 1`
- `n / 10` ha una cifra in meno: dopo al più dieci chiamate si arriva sotto 10

```
/** Stampa v[i], v[i+1], ... in ordine inverso. */
public static void stampaInversa(int[] v, int i) {
    if (i < v.length) {
        stampaInversa(v, i + 1);
        System.out.print(v[i] + " ");
    }
}

/** Stampa tutto v al contrario: parte da 0. */
public static void stampaInversa(int[] v) {
    stampaInversa(v, 0);
    System.out.println();
}
```

- il problema "più piccolo" è la parte di array da `i` in poi: `i` cresce fino a `v.length`, dove la parte è vuota e non c'è nulla da fare (caso base **implicito**: l'`if` è falso)
- prima la chiamata, **poi** la stampa: `v[0]` viene stampato per ultimo, quando tutte le chiamate successive sono già tornate
- due metodi in **overloading** (lezione 3): il client chiama quello senza indice

Commentiamo insieme – `fibonacci` e `main`

```java
/** L'n-esimo numero di Fibonacci. */
public static int fibonacci(int n) {
    chiamate++;
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

public static void main(String[] args) {
    System.out.println(sommaCifre(1234));
    int[] v = { 3, 8, 1, 6 };
    stampaInversa(v);
    System.out.println(fibonacci(20));
    System.out.println(chiamate + " chiamate");
}
```

```
10
6 1 8 3
6765
21891 chiamate
```

Rispondiamo insieme:

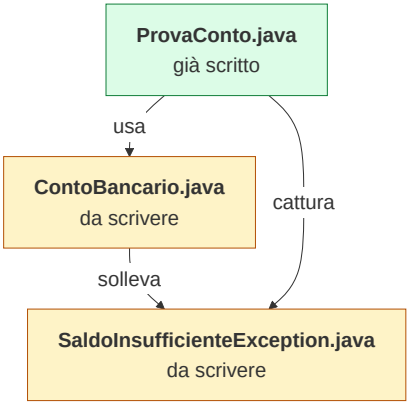
1. in `sommaCifre`, il caso base potrebbe essere `n == 0`, con `return 0`? Che cosa cambierebbe?
2. in `stampaInversa`, che cosa stamperebbe il metodo scambiando la chiamata ricorsiva e la `print`?
3. `fibonacci` ha **due** casi base in un solo `if`: quali? Perché ne servono due?
4. per F_{20} servono 21 891 chiamate: perché così tante? Guardate l'albero di F_4. Come lo calcolereste con un ciclo e due variabili?
5. perché `chiamate` è una variabile `static` della classe e non una variabile locale di `fibonacci`?

Insieme, passo passo (30 min) – il problema

Il problema. Un conto bancario deve **rifiutare** le operazioni impossibili e dire al client **perché**: un importo non positivo è un errore di chi scrive il programma; un prelievo più grande del saldo è una situazione normale, che il programma deve prevedere, e il client deve poter sapere **quanto manca**. Scriviamo `ContoBancario` e l'eccezione `SaldoInsufficienteException`. Il programma di prova `ProvaConto` è già scritto: legge l'importo da prelevare dalla tastiera, insistendo finché non arriva un numero positivo.

Le regole

- un conto ha un intestatario (che non cambia) e un saldo, che parte da 0; si leggono con due `get`
- `versa(importo)` e `preleva(importo)`: con un importo non positivo, nessuna modifica e un'eccezione con il messaggio `non positivo: -5.0`
- `preleva` con un importo maggiore del saldo: nessuna modifica e un'eccezione che dice quanto manca
- `trasferisci(altro, importo)`: un prelievo da questo conto e un versamento su `altro`; se il prelievo fallisce, nessun versamento
- `toString`: `Anna: saldo 100.0`



Insieme, passo passo – ProvaConto.java (1/2): l'input robusto

```java
/** Legge un importo positivo: insiste. */
public static double leggiImporto(Scanner in) {
    double importo = 0;
    while (importo <= 0) {
        System.out.print("Importo: ");
        try {
            importo = in.nextDouble();
        } catch (InputMismatchException e) {
            in.nextLine();   // scarta l'input
            System.out.println("non e' un numero");
        }
    }
    return importo;
}
```

```
Importo: abc
non e' un numero
Importo: -5
Importo: 250
```

- **input robusto**: un programma che legge dalla tastiera non deve terminare con uno stack trace perché l'utente ha scritto una parola al posto di un numero; deve chiedere di nuovo
- se `nextDouble` trova `abc`, solleva `InputMismatchException` e **lascia** `abc` nell'input: `nextLine()` lo scarta. Senza quella riga il ciclo rileggerebbe `abc` all'infinito
- `importo` resta a 0 e il ciclo ripete; un numero non positivo, come `-5`, è letto ma non fa uscire dal ciclo: qui basta la condizione del `while`, senza eccezioni
- il `catch` sta **dentro** il ciclo: dopo un errore si riprova; fuori dal ciclo, al primo errore il ciclo finirebbe

Insieme, passo passo – ProvaConto.java (2/2): il main

```
Scanner in = new Scanner(System.in);
ContoBancario anna = new ContoBancario("Anna");
ContoBancario luca = new ContoBancario("Luca");
anna.versa(100);
try {
    anna.preleva(leggiImporto(in));
    System.out.println("prelievo eseguito");
} catch (SaldoInsufficienteException e) {
    System.out.println("mancano " + e.getMancante());
} finally {
    System.out.println(anna);
}
try {
    anna.trasferisci(luca, 70);
} catch (SaldoInsufficienteException e) {
    System.out.println(e.getMessage());
}
System.out.println(anna + ", " + luca);
```

Con abc, -5, 250 (dopo i prompt):

```
mancano 150.0
Anna: saldo 100.0
Anna: saldo 30.0, Luca: saldo 70.0
```

Con 40:

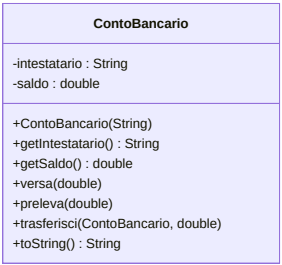
```
Importo: 40
prelievo eseguito
Anna: saldo 60.0
saldo insufficiente, mancano 10.0 euro
Anna: saldo 60.0, Luca: saldo 0.0
```

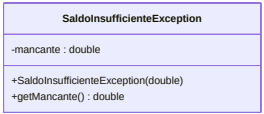
- il primo `catch` usa `getMancante()`, un metodo della **nostra** eccezione; il `finally` stampa il conto in entrambi i casi
- `IllegalArgumentException` non viene catturata: un importo non positivo è un errore del programma, che si evita a monte (qui `leggiImporto` accetta solo importi positivi), non si gestisce

Insieme, passo passo – le decisioni

Prima di scrivere il codice, decidiamo insieme:

1. **importo non positivo?** → `IllegalArgumentException`, non controllata: è una precondizione violata, il client la evita controllando l'importo (come fa `leggiImporto`)
2. **saldo insufficiente?** → un'eccezione nostra, **controllata** (`extends Exception`): dipende dallo stato del conto, il client deve prevederla e il compilatore glielo ricorda
3. **che cosa porta con sé?** → quanto manca: una variabile di istanza `final` con il suo `get`; il messaggio, costruito nel costruttore, con `super(...)`
4. **le intestazioni?** → `versa` e `preleva` sono `void`: l'esito non è più un `boolean`, chi chiama sa che l'operazione è riuscita se non arriva un'eccezione. Solo `preleva` e `trasferisci` dichiarano `throws`
5. **in che ordine i controlli di `preleva`?** → prima l'importo, poi il saldo; il saldo si modifica solo dopo entrambi
6. `trasferisci`**?** → chiama `preleva`, poi `altro.versa`: se `preleva` solleva l'eccezione, `trasferisci` termina lì e il versamento non avviene. Non la cattura: la rimanda al client, che sa che cosa fare





Una possibile soluzione (1/2) – l'eccezione e l'inizio del conto

```java
/** Sollevata se il saldo non basta. */
public class SaldoInsufficienteException
        extends Exception {

    private final double mancante;

    public SaldoInsufficienteException(
            double mancante) {
        super("saldo insufficiente, mancano "
                + mancante + " euro");
        this.mancante = mancante;
    }

    public double getMancante() {
        return mancante;
    }
}
```

- `extends Exception`: controllata. Il costruttore riceve quanto manca, costruisce il messaggio per `getMessage()` e lo passa a `super(...)`, di norma la prima istruzione
- `mancante` è `final`: un'eccezione descrive un fatto avvenuto, non cambia

```java
/** Un conto: errori con le eccezioni. */
public class ContoBancario {

    private final String intestatario;
    private double saldo;

    public ContoBancario(String intestatario) {
        this.intestatario = intestatario;
        saldo = 0;
    }

    public String getIntestatario() {
        return intestatario;
    }

    public double getSaldo() {
        return saldo;
    }
```

Una possibile soluzione (2/2) – ContoBancario.java

```java
/** Versa importo, che deve essere > 0. */
public void versa(double importo) {
    if (importo <= 0) {
        throw new IllegalArgumentException(
                "non positivo: " + importo);
    }
    saldo += importo;
}

/** Preleva importo, se il saldo basta. */
public void preleva(double importo)
        throws SaldoInsufficienteException {
    if (importo <= 0) {
        throw new IllegalArgumentException(
                "non positivo: " + importo);
    }
    if (importo > saldo) {
        throw new SaldoInsufficienteException(
                importo - saldo);
    }
    saldo -= importo;
}
```

```java
/** Sposta importo su altro. */
public void trasferisci(ContoBancario altro,
                        double importo)
        throws SaldoInsufficienteException {
    preleva(importo);
    altro.versa(importo);
}

@Override
public String toString() {
    return intestatario + ": saldo " + saldo;
}
```

- in `preleva` i due `throw` lasciano il saldo com'era: si modifica solo all'ultima riga
- `versa` non dichiara nulla: `IllegalArgumentException` è non controllata
- `trasferisci` non cattura: se `preleva` la solleva, l'eccezione attraversa `trasferisci` e arriva al client; `altro.versa` non viene eseguito

Riepilogo: cosa abbiamo trattato

Concetto (unità)	**Dove compare oggi**
Eccezioni a tempo di esecuzione, stack trace (U19)	`Divisione` con `7 0`; `Catena`; `SenzaBase`
`try` e `catch`, più `catch`, multi-catch, `finally` (U19)	`Divisione v2`, `DivisioneFinally`; `ProvaConto`
`throw`; stack unwinding; rilancio (U19)	`Media`; `Catena`, `CatenaRilancio`; `trasferisci`
Gerarchia; controllate e non; `throws` (U19)	`ProvaClone`; `preleva`, `trasferisci`
Le nostre eccezioni; quando non usare le eccezioni (U19)	`ParcheggioPienoException`, `SaldoInsufficienteException`; `Quoziente`
Input robusto con `Scanner` (U19)	`leggiImporto` in `ProvaConto`
Ricorsione: caso base, caso ricorsivo, convergenza (U21)	`fattoriale`; `sommaCifre`, `stampaInversa`, `fibonacci`
Lo stack delle chiamate; terminazione e profondità (U21)	il diagramma di `fattoriale(3)`; `StackOverflowError`
Ricorsione e iterazione; quando usare la ricorsione (U21)	`ContoAllaRovescia`; le 21 891 chiamate di `fibonacci(20)`

Prossima lezione (L11): la ricerca binaria, iterativa e ricorsiva, e quanto costa; la programmazione generica – classi e metodi generici, vincoli con `extends`, interfacce generiche, il carattere jolly `?`.

Homework 10 (da svolgere entro la prossima lezione)

Tre esercizi, ciascuno con il suo programma di prova, nella cartella `homework/`. Nei primi due, niente cicli: solo ricorsione.

1. `Hanoi.java` (con `ProvaHanoi.java`): la **Torre di Hanoi**. Tre paletti `A`, `B`, `C`; su `A` `n` dischi dal più grande (in basso) al più piccolo; vanno portati su `C` spostando un disco alla volta, mai uno più grande sopra uno più piccolo. Per spostare `n` dischi da `A` a `C`: si spostano `n-1` dischi su `B`, il disco `n` su `C`, gli `n-1` da `B` su `C`. `Hanoi` è un modulo con il metodo `static sposta(int n, char da, char a, char appoggio)`, che stampa ogni mossa (`disco 1: A -> C`), e un contatore `private static` delle mosse con `getMosse()` e `azzera()`. `ProvaHanoi` legge `n` da `args[0]`: se l'argomento manca stampa come si usa (con un `if`); se non è un intero, `Integer.parseInt` solleva `NumberFormatException` (non controllata): catturatela. Se $n < 1$, un messaggio; altrimenti alla fine stampa le mosse e le attese, $2^n - 1$.
2. `Stringhe.java` (con `ProvaStringhe.java`): un modulo con tre metodi `static` ricorsivi, che usano solo `length`, `charAt` e `substring`: `inverti(String s)` (`"ricorsione"` → `"enoisrocir"`), `isPalindroma(String s)` (primo e ultimo carattere uguali e la parte in mezzo palindroma) e `conta(char c, String s)` (quante volte `c` compare in `s`).
3. `Distributore.java` (con `CreditoInsufficienteException.java`, `EsauritoException.java` e `ProvaDistributore.java`): un distributore di un solo prodotto: costruttore `Distributore(String prodotto, double prezzo, int pezzi)`, credito iniziale 0. `inserisci(double monete)` solleva `IllegalArgumentException` se le monete non sono positive; `eroga()` consegna un pezzo e scala il prezzo dal credito, oppure solleva `EsauritoException` (pezzi finiti, controllato per primo) o `CreditoInsufficienteException` (con `getMancante()`), entrambe controllate; `restituisci()` azzera il credito e lo restituisce; `toString`. `ProvaDistributore` ha un metodo `static provaErogazione(Distributore d)` che chiama `eroga` e gestisce le due eccezioni con due `catch` diversi; il `main` prova un credito insufficiente, due erogazioni riuscite, il prodotto esaurito, il resto e un inserimento negativo.

Domande?

mattia.fonisto@unina.it